LangChain & Agentic AI – From Basics to Advanced 17 notes

RELIABLE STRUCTURED OUTPUT & REAL-WORLD APPLICATIONS

---

2. What Exactly Does with_structured_output() Do?

---

structured_llm = llm.with_structured_output(Student)

A beginner may think:

Sir, did we modify the original llm object?

No.

with_structured_output() creates a new Runnable/model wrapper configured to return output matching the supplied schema.

So:

llm

↓

Normal Chat Model

structured_llm

↓

Model Wrapper Configured for Structured Output

```
import os

from langchain_openai import ChatOpenAI
from pydantic import BaseModel

class Student(BaseModel):
    name: str
    age: int
    course: str

llm = ChatOpenAI(
    model="gpt-5.6-luna",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
)

structured_llm = llm.with_structured_output(Student)

print(type(llm))
print(type(structured_llm))
```

```
E:\lcclasses>py test.py
<class 'langchain_openai.chat_models.base.ChatOpenAI'>
<class 'langchain_core.runnables.base.RunnableSequence'>
```

---

3. Prove Original Model Still Works Normally

```
import os

from langchain_openai import ChatOpenAI
from pydantic import BaseModel
```

```
class Student(BaseModel):
    name: str
    age: int
    course: str

llm = ChatOpenAI(
    model="gpt-5.6-luna",
    api_key=os.getenv("OPENAI_API_KEY")
)

structured_llm = llm.with_structured_output(Student)

normal_result = llm.invoke(
    "Explain Python in one sentence."
)

structured_result = structured_llm.invoke(
    "Ravi is 22 years old and learning Python."
)
```

```
print("Normal Result Type:")
print(type(normal_result))

print("\nStructured Result Type:")
print(type(structured_result))
```

Concept:

llm.invoke()

↓

AIMessage

structured_llm.invoke()

↓

Student Object

---

include_raw field:

==========

By using this field we can specify whether original LLM response(raw) required to be included in the result or not.

Default value is False.

By default:

```
llm.with_structured_output(Student)
```

means:

```
llm.with_structured_output(
    Student,
    include_raw=False
)
```

With a Pydantic schema, if everything succeeds, we directly receive the parsed Pydantic object. If parsing fails, the parsing error is raised.

Model Response

↓

Parse + Validate

↓

Success?

↙ ↘

Yes No

↓ ↓

Student Error

Object Raised

6. Why Would We Want the Raw Response?

For debugging purposes

Suppose something goes wrong.

We may want to know:

- What did the model actually return?
- Was parsing successful?
- What was the parsed object?
- Was there a parsing error?

For debugging, this information is very useful.

LangChain provides:

include_raw=True

7. Using include_raw=True

Example:

```
structured_llm = llm.with_structured_output(
    Student,
    include_raw=True
)
```

Now the returned result is different.

Instead of directly getting:

Student Object

we get a dictionary containing:

raw

parsed

parsing_error

Current LangChain defines these three keys when include_raw=True.

10. Understand raw

What is:

result["raw"]

It is the underlying model response.

Normally it is a message object such as an:

AIMessage

So:

raw

↓

Original model-side response

This is useful when debugging.

11. Understand parsed

What is:

result["parsed"]

If everything succeeds:

parsed

↓

Student Object

Example:

student = result["parsed"]

print(student.name)

print(student.age)

print(student.course)

Very useful.

12. Understand parsing_error

Now:

result["parsing_error"]

If parsing succeeds:

None

If parsing fails:

Error Information

Therefore:

parsing_error = None

↓

Parsing successful

==========

13. Complete include_raw Demonstration

```
import os

from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field

class Employee(BaseModel):

    name: str = Field(
        description="Name of the employee"
    )

    technology: str = Field(
        description="Primary technology"
    )
```

```
    experience: int = Field(
        description="Professional experience in years"
    )

llm = ChatOpenAI(
    model="gpt-5.6-luna",
    api_key=os.getenv("OPENAI_API_KEY")
)

structured_llm = llm.with_structured_output(
    Employee,
    include_raw=True
)

result = structured_llm.invoke(
    "Ramesh is a Java developer with 6 years of experience."
)

print("\n---------- RAW ----------")
print(result["raw"])
```

```
print("\n---------- PARSED ----------")
print(result["parsed"])

print("\n---------- PARSING ERROR ----------")
print(result["parsing_error"])
```

==========

47. Complete Reliable Application with include_raw=True

```
import os

from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI

class Student(BaseModel):

    name: str = Field(
        description="Name of the student"
    )
```

```
    age: int | None = Field(
        default=None,
        description="Student age if explicitly provided"
    )

    technology: str | None = Field(
        default=None,
        description="Technology the student is learning"
    )

    experience: int | None = Field(
        default=None,
        description="Experience in years if explicitly provided"
    )

llm = ChatOpenAI(
    model="gpt-5.6-luna",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
structured_llm = llm.with_structured_output(
    Student,
    include_raw=True
)

text = input(
    "Enter information: "
)

prompt = f"""
Extract student information.

Rules:
1. Use only the supplied text.
2. Do not invent information.
3. If optional information is unavailable, use null.

Text:
{text}
"""
```

```
result = structured_llm.invoke(prompt)

if result["parsing_error"] is None:

    student = result["parsed"]

    print("\nParsing Successful")

    print("\nName:")
    print(student.name)

    print("\nAge:")
    print(student.age)

    print("\nTechnology:")
    print(student.technology)

    print("\nExperience:")
    print(student.experience)
```

```
else:

    print("\nParsing Failed")

    print(result["parsing_error"])

    print("\nRaw Model Response:")

    print(result["raw"])
```

---

49. include_raw=False vs include_raw=True

Feature	False	True
Default?	Yes	No
Parsed result	Returned directly	Inside parsed
Raw model response	No	raw
Parsing error	Raised	Captured in parsing_error
Useful for simple app	Yes	Yes
Useful for debugging	Less information	Very useful

---

29. Restricted Values with Literal

==========

Suppose sentiment must be only:

positive

negative

neutral

We can use:

from typing import Literal

Then:

```
sentiment: Literal[
    "positive",
    "negative",
    "neutral"
]
```

Now our schema expresses the allowed categories clearly.

30. First Literal Example

```
from pydantic import BaseModel
from typing import Literal

class Sentiment(BaseModel):

    sentiment: Literal[
        "positive",
        "negative",
        "neutral"
    ]
```

Instead of arbitrary values like:

Very Good

Happy

Amazing

Badish

our application expects one of:

positive

negative

neutral

31. Numeric Validation Rules

Pydantic Field() can also define constraints.

Example:

```
confidence: float = Field(
    ge=0,
    le=1,
    description="Confidence score between 0 and 1"
)
```

Meaning:

ge = greater than or equal to

le = less than or equal to

Therefore:

0 ≤ confidence ≤ 1

32. Sentiment Schema

```
from typing import Literal
from pydantic import BaseModel, Field
```

```
class SentimentResult(BaseModel):

    sentiment: Literal[
        "positive",
        "negative",
        "neutral"
    ] = Field(
        description="Overall sentiment"
    )

    confidence: float = Field(
        ge=0,
        le=1,
        description="Confidence score from 0 to 1"
    )

    reason: str = Field(
        description="Short reason for the classification"
    )
```

Now our application gets predictable categories.

---

44. Complete Sentiment Analyzer

```
import os

from typing import Literal

from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field

class SentimentResult(BaseModel):

    sentiment: Literal[
        "positive",
        "negative",
        "neutral"
    ] = Field(
        description="Overall sentiment of the text"
    )
```

```
    confidence: float = Field(
        ge=0,
        le=1,
        description="Confidence score between 0 and 1"
    )

    reason: str = Field(
        description="Very short reason for the sentiment"
    )

llm = ChatOpenAI(
    model="gpt-5.6-luna",
    api_key=os.getenv("OPENAI_API_KEY")
)

structured_llm = llm.with_structured_output(
    SentimentResult
)

text = input(
```

```
    "Enter customer feedback: "
)

result = structured_llm.invoke(text)

print("\nSENTIMENT ANALYSIS")
print("-----------------")

print("Sentiment  :", result.sentiment)
print("Confidence :", result.confidence)
print("Reason     :", result.reason)
```

==========

45. Why Structured Output Is Excellent Here

Without structured output:

```
The customer appears to be very happy...
But our software may need:

sentiment = positive
```

confidence = 0.95

Then our program can write:

```
if result.sentiment == "negative":
    print("Send feedback to support team")
```

Now the AI result can directly participate in application logic.

46. Structured Output → Business Logic

This is very important.

User Feedback

↓

LLM

↓

Structured Result

↓

Python Condition

↓

Business Action

Example:

```
if result.sentiment == "negative":
```

```
    print("Escalate to support team")

else:
    print("No escalation required")
```

This is where structured output becomes very powerful.

==========

```
import os

from typing import Literal

from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field


class SupportTicket(BaseModel):

    category: Literal[
        "billing",
        "technical",
        "account",
```

```
    "other"
] = Field(
    description="Main category of the customer issue"
)

priority: Literal[
    "low",
    "medium",
    "high"
] = Field(
    description="Priority level of the issue"
)

severity: int = Field(
    ge=1,
    le=5,
    description="Severity from 1 to 5"
)

summary: str = Field(
```

```
        description="Very short summary of the customer issue"
    )


llm = ChatOpenAI(
    model="gpt-5.6-luna",
    api_key=os.getenv("OPENAI_API_KEY")
)


structured_llm = llm.with_structured_output(
    SupportTicket
)


text = input(
    "Enter customer issue: "
)
```

```
result = structured_llm.invoke(text)



print("\nSUPPORT TICKET")
print("------------------")

print("Category :", result.category)
print("Priority :", result.priority)
print("Severity :", result.severity)
print("Summary  :", result.summary)
```

input1:

My payment was deducted twice from my credit card.

Please refund the extra amount.

input2:

I forgot my password and cannot login to my account.